

الاسم: محمد جوده محمد حافظ

السكشن: 4

Summer Training

Introducing Generative AI with AWS

Introducing Generative AI with AWS

Introduction: Unlocking the creative power of AI:

From drafting compelling marketing copy and generating lifelike images to solving complex code challenges, generative AI is rapidly reshaping industries and redefining the boundaries of human-computer collaboration. However, for many professionals and enthusiasts, the question remains: how does one transition from being a passive observer to an active builder in this new AI-driven era? The "Introducing Generative AI with AWS" course is designed to answer this very question. This report documents the journey through this foundational program, which aims to bridge the gap between theoretical understanding and practical implementation. By leveraging the robust, cloud-native ecosystem of Amazon Web Services, the course provides a structured path to not only understand the mechanics of large language models and diffusion models but also to apply them responsibly. The following sections will break down the course structure, highlight key takeaways, and assess its effectiveness in building a solid foundation for future exploration and innovation in generative AI.

We will discuss all the steps of making a model and implementing a pipeline from framing the problem to the deployment of the model and what to do in it.

1- Framing the problem:

with more information about the problem, you can begin to design an approach.

First, can a machine learning solve the problem, or would a traditional approach make more sense?

Second, is this problem a supervised or unsupervised machine learning problem? Do you have labeled data to train a supervised model?

You have many questions to ask yourself and the business. Ultimately, You should try to validate the use of machine learning and confirm that you have

access to the right people and data. Then, devise the simplest solution to the problem.

2-Data Considerations:

When choosing your data, you must consider several factors.

Additional questions to ask yourself include the following:

- a. What data do you need to address the problem?
- b. How much data do you have, and where is it?
- c. Do you have access to that data?
- d. Can you bring this data into one centralized repository?
- e. Is the data representative.

1. Get an understanding of the data

- a. You have further formulated the problem, but what data do you need to actually train the model to reach the desired output and intended business outcome? Do you have access to that data? If so, how much data do you have and where is it?
- b. What solution can you use to bring all of this data into one centralized repository? These questions are essential to answer at this stage
- c. To answer these questions, it's a good idea to start with an overview of the typical elements of an ML dataset.

2. Get a domain expert:

- a. Your data should be representative of the data that you will have when you are using the model to make prediction. What I mean by this is this "every choice that is available, your data should include them to enable the model to learn the less happening choice, but it must happen in the data.
- b. Given what you know about the elements of an ML dataset, recall one of the original questions. **Which data do you need to actually train the model to reach the desired output and intended business outcome?**
At this stage in the ML pipeline, it's vital to get domain expertise. With domain knowledge, you can begin to determine the features and target data that your model needs to make accurate predictions.

3. Evaluate the quality of your data too:

- a. A general rule in this case is that good data contains a signal about the phenomenon that you're trying to model.
- b. What I mean here is that each feature that helps you in predicting or helps you to reach the labeled data must be in the data and must be representative.

4. Start identifying features and labels you already have:

- a. Your data is made up of two elements: Features and labels. A feature is an attribute that can be used to help identify patterns and predicts future answers. A feature in the credit card example could be the date of transaction, vendor, or amount of the transaction, in dollars.
- b. Subsequently, data for which you already know the answer is called labeled data. The labeled data is the answer that you want the model to predict, so in the credit card example, the label of any given observation is either "fraud" or "not fraud"

5. Do you need a lot of labeled data?

- a. Many of the examples that you've seen so far represent supervised learning and therefore include labeled data. But what if you don't have labeled data? In some cases, getting the labeled data that you need can be arduous.
- b. Consider the use of autonomous driving. Image data must be precisely labeled with where exactly the car is at any given moment. Labels also must include where the car might be relative to some other object like the road, a building, Without the labeling, the trained model would not be as accurate and could lead to potentially fatal mistakes.
- c. Feature engineering is dealing with your data to make it usable. It is the process of selecting or creating the features that you will use to train your model. Features are the columns of data that you have within your dataset.

3. Preparing data:

The data that you gather won't be ready to be used. The data might be dirty, which means that it contains errors or duplicate data. Dirty data has possibly been collected by using the wrong method or population for your business problem or has some other data integrity issue. Using dirty data in your machine learning models will lead to incorrect results or an inability to run the model effectively.

For this reason, the data that you have gathered must be prepared for use in the machine learning model. There are four considerations for preparing your data.

1. Encoding Data:

Most ML algorithms work best with numerical data. Therefore, you must make sure that all columns in your dataset contain numeric data by converting or encoding the data.

You might need to make several passes through the dataset before you can encode it. For example, you might have variability in the text values(for example, some rows might contain both medium and med).

If the categorical data has order to it, you can encode the text into numerical values that capture this ordinal relationship. For data that show maintenance costs, you might encode low to 1, medium to 2, high to 3 and very high to 4

Tools are available to encode your categorical data after you make sure that it is all uniform.

2. Cleaning data:

You must make sure that the data is clean from all noise and to make all the features clean in the training set and the test set and make sure that the test features are clean.

Note that clean mean to make the column either for feature or label in more uniform values not may be two different values for indenting one value and also from any bad noisy in the data.

3. Finding missing data:

Missing can make it difficult to accurately interpret the relationship between the related feature and the target variable. Thus, regardless of what caused the data to be missed, it is important that you deal with issue.

Unfortunately, most ML algorithms can't deal with missing values automatically. You must update missing values with something meaningful and relevant to the problem.

4. Handling outliers:

You might also need to clean your data because of any outliers that might exist. They are not always something that you want to clean up, because they can add richness to your dataset. However, they can also make it more difficult to make accurate predictions.

You can think of outliers as falling into two broad categories. The two categories are a single variation for a single variable(univariate) and a variation of two or more variables(multivariate).

There're two features that more important, Feature selection and Feature Extraction:

Feature selection:

After you extract features, you must select the most appropriate features to train your model. Three main feature selection methods are available: filter, wrapper, and embedded.

- **Filter methods** use statistical methods to measure the relevance of features by their correlation with the target variable.
- **Wrapper methods** measures the usefulness of a subset of features by training a model on it and measuring the success of the model.
- **Embedded methods** are algorithm-specific and might use a combination of both.

Filter methods are faster and cheaper than wrapper methods because they don't involve training the models repeatedly. Wrappers typically find the best subset of features, with a risk of overfitting compared to using subsets of features from filter methods.

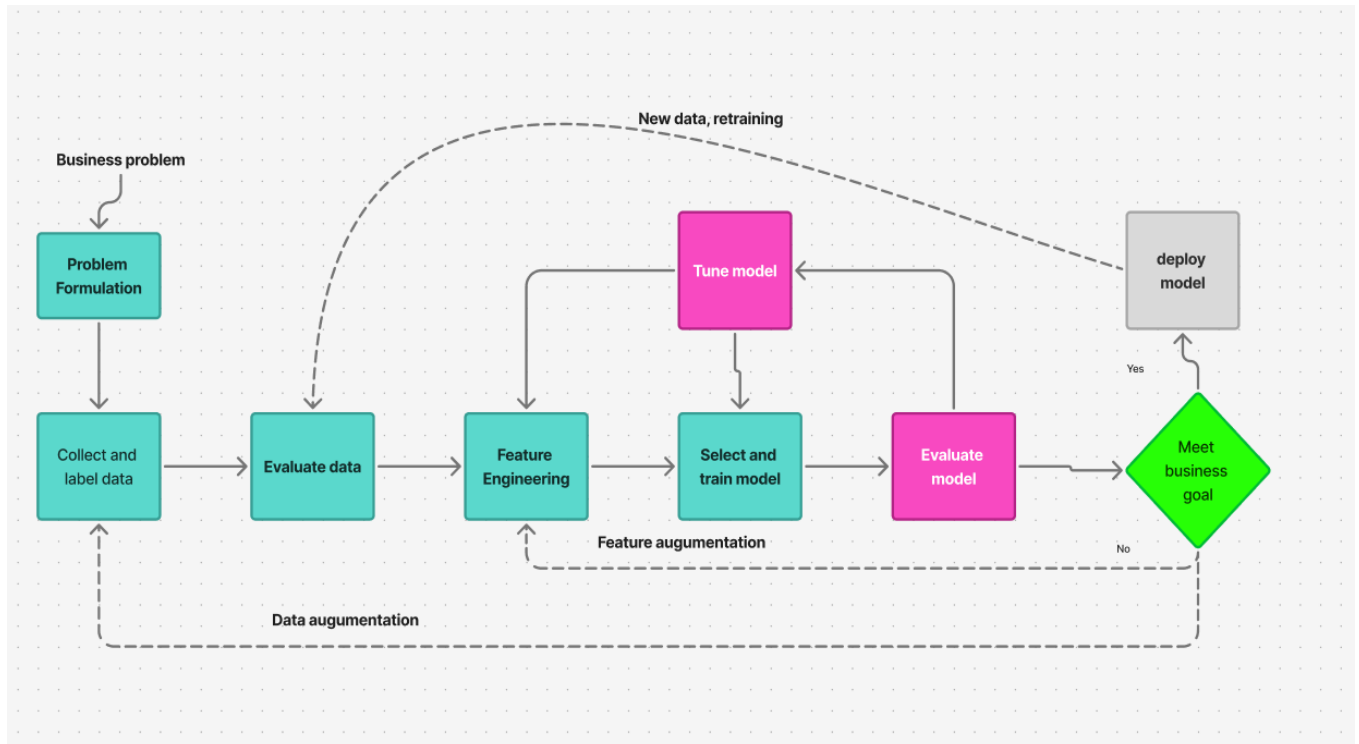
Feature extraction:

Feature extraction covers many activities that range from handling missing data to converting text data into numerical data. It should give you some idea of the data handling that's needed to get data into a useful state. You must make sure that the data is in the correct format, with consistent representations of data.

Common tasks include:

- Encoding ordinal and non-ordinal data.
- Finding missing data.
- Handling outliers in the data

Machine learning- Tune and evaluate model



IN the first pages we had discussed the steps of preparing and processing data that in the pipeline with names:

- Problem Formulation.
- Collect and label data.
- Evaluate data.
- Feature Engineering.
- Select and train model.

Now we will discuss the other 3 steps that are evaluate model and tune model and deploy model.

Evaluate and tune model:

After you train and tune your models and decide which model is the best for your business problem, you must evaluate your model. You want to determine whether it will perform well in predicting the target on new and future data.

During evaluation you are testing your model against the validation data you set aside. The validation data provides an unbiased evaluation of a model fit while tuning model.

The test data has known values and lets you assess the accuracy of the model after tuning. If the model performs well on the test data it will likely perform well on new data with unknown target values.

This is why it is important to set aside enough test data and not use that data for training or validation.

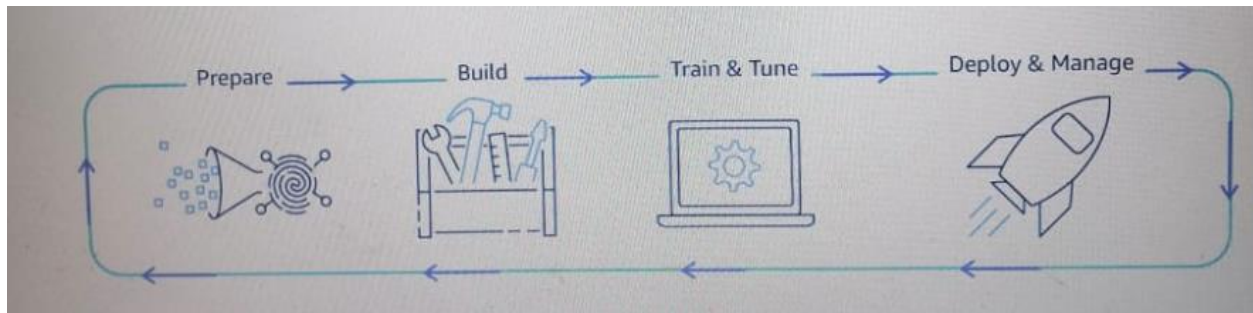
Remember the normal split is 80% for training, 10% for validation, and 10% for testing.

Model deployment:

After you have trained, tuned, and tested your model, you are ready to deploy it. To test and get performance metrics from your model, you must make inferences or predictions from the model—which typically requires deployment. Deployment for testing is different from production, although the mechanics are the same. These assessments can involve anything from a few requests to deployments that handle tens of thousands of requests.

Introducing Amazon SageMaker:

SageMaker helps data scientists and developers to prepare, build, train and deploy high-quality machine learning models. It also provides the ability to iterate the process until they get their model just right. The service quickly brings together a broad set of purpose-built capabilities for machine learning in one comprehensive service.



SageMaker notebooks:

Amazon SageMaker can deploy machine learning instances that run Jupyter notebooks and JupyterLabs.

Data visualization:

After you entered the data you have into Jupyter notebook, SageMaker can make visualizations for you, so that visualizing your data can help you select the features that you actually want to use when you build your model.

Model selection:

You can build a model in Amazon sage maker by using any of the following algorithms options:

- Use sage maker built-in algorithm
- Write a script in your framework.
- Get an algorithm from AWS Marketplace.
- Bring your own algorithm.

After training your model, when you're ready to improve(or tune) it, Amazon SageMaker provides automatic tuning. This feature is also called hyperparameter tuning or hyperparameter optimization and is about adjusting and tweaking your algorithm.

Project 1

I used Amazon SageMaker to make a food model, that generate a meal from any food that available for you in the home actually food and it will give you the recipe step by step.

Project 2:

I had make a supervised ML learning model using linear regression algorithm to Predict the prices of real estate properties in Boston and I had applied all the steps to create a model.

I get an RMSE of 75.63% in this model.